

13 variant (1.3) Callback

```
using System;using System.Threading; using System.Linq;
public class Program{ public static void Main() {
    var processor = new ClsA();    processor.StartProcessing(); } }
public class ClsA{
    private readonly Random _random = new Random();
    private int[] _sharedArray; private int _iterations;
    private int _completedIterations = 0;
    // Семафоры для управления потоками
    private readonly Semaphore _semB = new Semaphore(0, 1);
    private readonly Semaphore _semC = new Semaphore(0, 1);
    private readonly Semaphore _semA = new Semaphore(0, 2);
    // Для ожидания двух сигналов
    private ClsB _clsB; private ClsC _clsC;
    public ClsA() { _clsB = new ClsB(this);
        _clsC = new ClsC(this); _iterations = _random.Next(3, 7); }
    public void StartProcessing() {
        Console.WriteLine($"Starting processing with {_iterations}
iterations");
        new Thread(_clsB.Run).Start(); new Thread(_clsC.Run).Start();
        while (_completedIterations < _iterations) {
            GenerateArray(); Console.WriteLine($"\\nIteration
{_completedIterations + 1}: Generated array: {string.Join(" ",
_sharedArray)}");
            _semB.Release(); _semA.WaitOne(); // От ClsB
            _semA.WaitOne(); _completedIterations++; }
        _clsB.Stop(); _clsC.Stop(); _semB.Release(); _semC.Release();
        Console.WriteLine("\\nProcessing completed!"); }
    private void GenerateArray() {
        int size = _random.Next(5, 15); // 5-14 элементов
        _sharedArray = new int[size]; for (int i = 0; i < size; i++)
        { _sharedArray[i] = _random.Next(0, 100); } }
    public int[] GetSharedArray() => _sharedArray;
    public Semaphore GetSemaphoreB() => _semB;
    public Semaphore GetSemaphoreC() => _semC;
    public void CallbackFromB(int[] sortedLowerPart) {
        Console.WriteLine($"Callback from B: Sorted lower part:
{string.Join(" ", sortedLowerPart)}"); _semA.Release(); }
    public void CallbackFromC(int[] sortedUpperPart){
        Console.WriteLine($"Callback from C: Sorted upper part:
{string.Join(" ", sortedUpperPart)}"); _semA.Release(); } }
public class ClsB{ private readonly ClsA _clsA;
    private bool _isRunning = true;
    public ClsB(ClsA clsA) { _clsA = clsA; }
    public void Run() { while (_isRunning) {
        _clsA.GetSemaphoreB().WaitOne();if (! _isRunning) break;
        var array = _clsA.GetSharedArray();
        var sorted = array.OrderBy(x => x).ToArray(); double median;
        if (sorted.Length % 2 == 0)
            median = (sorted[sorted.Length/2 - 1]
+sorted[sorted.Length/2]) / 2.0;
        else
            median = sorted[sorted.Length/2];
```

```
Console.WriteLine($"B: Median is {median}");
        var lowerPart = array.Where(x => x < median).ToArray();
        var upperPart = array.Where(x => x >=
median).ToArray();
        Array.Sort(lowerPart);
        _clsA.GetSemaphoreC().Release();
        _clsA.CallbackFromB(lowerPart); } }
    public void Stop() => _isRunning = false;}
public class ClsC{
    private readonly ClsA _clsA; private bool _isRunning = true;
    public ClsC(ClsA clsA) { _clsA = clsA; }
    public void Run() { while (_isRunning) {
        _clsA.GetSemaphoreC().WaitOne(); if (! _isRunning) break;
        var array = _clsA.GetSharedArray();
        var sorted = array.OrderBy(x => x).ToArray();
        double median; if (sorted.Length % 2 == 0)
            median = (sorted[sorted.Length/2 - 1] +
sorted[sorted.Length/2]) / 2.0;
        else
            median = sorted[sorted.Length/2];
        var upperPart = array.Where(x => x >=
median).ToArray();
        Array.Sort(upperPart);
        _clsA.CallbackFromC(upperPart); } }
    public void Stop() => _isRunning = false;
}
```

13. (2.3 //the IterationExctension class is the same only the iterator is changing and the code is also of it.

```
public class IteratorExample
{ static void Main() {
    var matrix = new List<List<double>>> {
        new List<double> {1.0, 1.1, 1.2},
        new List<double> {2.0, 2.1, 2.2},
        new List<double> {3.0, 3.1, 3.2}};
    // Итератор для последнего столбца (индекс 2) снизу вверх
    var iter = matrix.MakeCustomIterator(
        new int[] {matrix.Count - 1, 2}, // Начинаем с
последней строки, последнего столбца
        (coll, cur) => coll[cur[0]][cur[1]], // Получа
ем текущий элемент
        (cur) => cur[0] < 0, // Завершаем, когда вышли
за верхнюю границу
        (cur) => new int[] {cur[0] - 1, cur[1]}); // Дв
игаемся вверх по строкам
    foreach(var item in iter) {
        Console.WriteLine(item); } } }
```

13 (3.1)

Сборщик мусора определяет, что объект не нужен, когда он становится **недостижимым** из корневых ссылок (локальных переменных, статических полей, активных потоков). Это выявляется во время фазы маркировки, когда GC обходит граф объектов и помечает только достижимые объекты. Недостижимые объекты считаются мусором и удаляются в фазе очистки. Example:

```
class MyClass { public int Value;}
void Example() {
```

```
var obj = new MyClass(); // Создаётся объект (достигим)
obj.Value = 10;
obj = null; // Объект больше не достижим — GC удалит его }
В C# контроль нарушения границ массива при индексировании
указателя зависит от контекста:
```

1. **Управляемый код:** CLR автоматически проверяет индекс. Если индекс выходит за пределы (0 до `array.Length - 1`), выбрасывается `IndexOutOfRangeException`.

```
int[] array = new int[5];
try { int value = array[6]; // Вызовет IndexOutOfRangeException }
catch (IndexOutOfRangeException) {
    Console.WriteLine("Индекс вне границ!"); }
```

2. **Небезопасный код с указателями:** Проверки границ отсутствуют, ответственность на программисте. Неправильный индекс может привести к сбою.

```
unsafe { int[] array = new int[5] { 1, 2, 3, 4, 5 };
    fixed (int* ptr = array) { int index = 6;
        if (index >= 0 && index < array.Length)
            ptr[index] = 10; // Безопасный доступ
        else
            Console.WriteLine("Индекс вне границ!"); // Ручная проверка
        // ptr[6] = 10; // Ошибка: доступ за пределами массива,
        // возможен сбой
    }
}
```

Variant 13(without the callback starts)

13. (1.3 without the callback)

```
using System; using System.Threading; using System.Linq;
public class Program { public static void Main() {
    ClsA clsA = new ClsA(); clsA.StartProcessing(); }
public class ClsA {
    private readonly Random _random = new Random();
    private int[] _sharedArray; private int _iterations;
    private int _currentIteration = 0;
    // Семафоры для синхронизации
    private readonly Semaphore _semB = new Semaphore(0, 1);
    private readonly Semaphore _semC = new Semaphore(0, 1);
    private readonly Semaphore _semA = new Semaphore(0, 1);
    private ClsB _clsB; private ClsC _clsC;
    public ClsA() { _clsB = new ClsB(this); _clsC = new ClsC(this);
        _iterations = _random.Next(3, 7); // 3-6 итераций }
    public void StartProcessing() {
        Console.WriteLine($"Starting processing with {_iterations}
iterations");
        // Запускаем потоки
        new Thread(_clsB.Run).Start(); new Thread(_clsC.Run).Start();
        for (_currentIteration = 0; _currentIteration < _iterations;
        _currentIteration++){ GenerateArray(); Console.WriteLine($"Iteration
{_currentIteration + 1}: Generated array: {string.Join(", ",
_sharedArray)}");
        // Разрешаем ClsB начать работу
        _semB.Release();
        // Ждем сигналов от обоих классов
        _semA.WaitOne(); // От ClsB _semA.WaitOne(); // От ClsC
        // Небольшая пауза между итерациями
        Thread.Sleep(500); }
    // Завершаем работу потоков
```

```
_clsB.Stop(); _clsC.Stop(); _semB.Release(); _semC.Release(); }
private void GenerateArray() { int size = _random.Next(5, 15);
    _sharedArray = new int[size]; for (int i = 0; i < size; i++) {
        _sharedArray[i] = _random.Next(0, 100); } }
public int[] GetSharedArray() => _sharedArray;
public Semaphore GetSemaphoreB() => _semB;
public Semaphore GetSemaphoreC() => _semC;
public void SignalFromB(int[] sortedLowerPart) {
    Console.WriteLine($"Received from B: {string.Join(", ",
sortedLowerPart)}"); _semA.Release(); }
public void SignalFromC(int[] sortedUpperPart) {
    Console.WriteLine($"Received from C: {string.Join(", ",
sortedUpperPart)}"); _semA.Release(); }
public class ClsB { private readonly ClsA _clsA;
    private bool _isRunning = true;
    public ClsB(ClsA clsA) { _clsA = clsA; }
    public void Run() { while (_isRunning) {
        _clsA.GetSemaphoreB().WaitOne(); if (!_isRunning) break;
        var array = _clsA.GetSharedArray();
        // Находим медиану
        var sorted = array.OrderBy(x => x).ToArray();
        double median = sorted.Length % 2 == 0 ? (sorted[sorted.Length/2 -
1] + sorted[sorted.Length/2]) / 2.0 : sorted[sorted.Length/2];
        Console.WriteLine($"B: Median is {median}");
        // Разделяем массив
        var lowerPart = array.Where(x => x < median).ToArray();
        var upperPart = array.Where(x => x >= median).ToArray();
        Array.Sort(lowerPart); _clsA.GetSemaphoreC().Release();
        _clsA.SignalFromB(lowerPart); } }
    public void Stop() => _isRunning = false; }
public class ClsC { private readonly ClsA _clsA;
    private bool _isRunning = true;
    public ClsC(ClsA clsA) { _clsA = clsA; }
    public void Run() { while (_isRunning) {
        _clsA.GetSemaphoreC().WaitOne();
        if (!_isRunning) break;
        var array = _clsA.GetSharedArray();
        var sorted = array.OrderBy(x => x).ToArray();
        double median = sorted.Length % 2 == 0 ? (sorted[sorted.Length/2 -
1] + sorted[sorted.Length/2]) / 2.0 : sorted[sorted.Length/2];
        var upperPart = array.Where(x => x >= median).ToArray();
        Array.Sort(upperPart); _clsA.SignalFromC(upperPart); } }
    public void Stop() => _isRunning = false; }
```

13 (2.4)

Проблема в том, что интерфейс Shape заставляет все классы-фигуры реализовывать методы для всех типов фигур, даже если они им не нужны.

// Разделяем интерфейсы для каждой фигуры

```
interface ICircle { void Draw(); }
interface ISquare { void Draw(); }
interface IRectangle { void Draw(); }
class Circle : ICircle { public void Draw() {
    // Реализация рисования круга } }
class Square : ISquare { public void Draw() {
    // Реализация рисования квадрата } }
class Rectangle : IRectangle { public void Draw() {
    // Реализация рисования прямоугольника } }
```

Or like this:

Если нужно сохранить полиморфизм фигур:

```
interface IShape { void Draw(); }
class Circle : IShape { public void Draw() {
    // Реализация для круга }}
class Square : IShape { public void Draw() {
    // Реализация для квадрата }}
class Rectangle : IShape { public void Draw() {
    // Реализация для прямоугольника }}
```

13. (The other two are the same as in the first page.)

Varaint 2

2 (1.2)

```
using System; using System.Threading; using System.Linq;
public class Program { public static void Main() {
    ClsA processor = new ClsA(); processor.StartProcessing(); }}
public class ClsA{
    private readonly Random _random = new Random();
    private int[] _sharedArray; private int _iterations;
    private int _currentIteration = 0;
    // Семафоры для управления потоками
    private readonly Semaphore _semB = new Semaphore(0, 1);
    private readonly Semaphore _semC = new Semaphore(0, 1);
    private readonly Semaphore _semA = new Semaphore(0, 2); // Для
    ожидания двух сигналов
```

```
    private ClsB _clsB; private ClsC _clsC;
    public ClsA() { _clsB = new ClsB(this);
        _clsC = new ClsC(this); _iterations = _random.Next(3, 7); }
    public void StartProcessing() {
        Console.WriteLine($"Starting processing with {_iterations}
iterations");
        new Thread(_clsB.Run).Start(); new Thread(_clsC.Run).Start();
        for (_currentIteration = 0; _currentIteration < _iterations;
        _currentIteration++) {
            GenerateArray(); Console.WriteLine($"Iteration
{_currentIteration + 1}: Generated array: {string.Join(", ",
        _sharedArray)}");
            _semB.Release(); _semC.Release();
            _semA.WaitOne(); _semA.WaitOne();
            Thread.Sleep(300); }
```

```
_clsB.Stop(); _clsC.Stop(); _semB.Release(); _semC.Release(); }
private void GenerateArray() {
    int size = _random.Next(5, 11); // 5-10 элементов
    _sharedArray = new int[size]; for (int i = 0; i < size; i++) {
        _sharedArray[i] = _random.Next(1, 10); } }
// Методы для доступа к общим ресурсам
public int[] GetSharedArray() => _sharedArray;
public Semaphore GetSemaphoreB() => _semB;
public Semaphore GetSemaphoreC() => _semC;
public void SignalFromB(double arithmeticMean) {
    Console.WriteLine($"Arithmetic mean from B:
{arithmeticMean:F2}"); _semA.Release(); }
public void SignalFromC(double geometricMean) {
    Console.WriteLine($"Geometric mean from C:
{geometricMean:F2}"); _semA.Release(); }}
public class ClsB{ private readonly ClsA _clsA;
```

```
private bool _isRunning = true;
public ClsB(ClsA clsA) { _clsA = clsA; }
public void Run() { while (_isRunning) {
    _clsA.GetSemaphoreB().WaitOne(); if (!_isRunning) break;
    var array = _clsA.GetSharedArray();
    double arithmeticMean = array.Average();
    _clsA.SignalFromB(arithmeticMean); } }
public void Stop() => _isRunning = false; }
public class ClsC{
    private readonly ClsA _clsA; private bool _isRunning = true;
    public ClsC(ClsA clsA) { _clsA = clsA; }
    public void Run() {
        while (_isRunning) { _clsA.GetSemaphoreC().WaitOne();
            if (!_isRunning) break;
            var array = _clsA.GetSharedArray();
            double product = 1; foreach (var num in array) {
                product *= num; }
            double geometricMean = Math.Pow(product, 1.0 / array.Length);
            _clsA.SignalFromC(geometricMean); } }
    public void Stop() => _isRunning = false; }
```

2 (2.1)

В данном коде нарушен принцип единственной ответственности (Single Responsibility Principle, SRP) из SOLID, а также принцип разделения интерфейсов (Interface Segregation Principle, ISP).

Проблемы в текущем коде:

Класс Animal имеет две ответственности:

Хранение данных о животном (getAnimalName)

Сохранение животного в БД (saveAnimal)

Нарушение SRP: класс должен отвечать только за представление животного, а не за его сохранение.

Потенциальное нарушение ISP: если будут создаваться другие классы животных, они будут вынуждены реализовывать метод сохранения, даже если он им не нужен. Solved shape:

// Класс отвечает только за представление животного

```
class Animal { constructor(private name: string) {}
    getAnimalName(): string { return this.name; } }
```

// Отдельный класс для работы с хранилищем

```
class AnimalRepository { saveAnimal(a: Animal): void {
    // Логика сохранения животного в БД } }
```

Or like this:

```
interface IAnimal { name: string; getAnimalName(): string; }
interface IAnimalRepository { saveAnimal(a: IAnimal): void; }
```

// Реализация животного

```
class Animal implements IAnimal {constructor(public name: string){}
    getAnimalName(): string { return this.name; } }
```

// Реализация хранилища

```
class AnimalRepository implements IAnimalRepository {
    saveAnimal(a: IAnimal): void { // Логика сохранения } }
```

2 (3.3)

Сборка мусора (Garbage Collection, GC) в управляемых языках, таких как C# (.NET), происходит в следующих случаях:

1. ****Нехватка памяти****: - Когда в куче (heap) недостаточно места для выделения нового объекта, CLR запускает GC для освобождения памяти.

2. ****Превышение порога поколения****: - В .NET объекты делятся на поколения (Gen 0, Gen 1, Gen 2). GC запускается, когда объем

объектов в определенном поколении превышает заданный порог (например, Gen 0 заполняется быстрее всего).

3. ****Явный вызов****: - Программист может вызвать `GC.Collect()` для принудительной сборки мусора, но это не рекомендуется, так как CLR лучше оптимизирует момент сборки.

4. ****Фоновая сборка****: - В .NET поддерживается фоновая сборка мусора (Background GC) для Gen 2, которая выполняется в отдельном потоке, минимизируя паузы приложения.

5. ****События системы****: - GC может запускаться при низкой активности приложения, выходе из определенных методов или при изменении состояния системы (например, при нехватке системной памяти). Пример

```
class Program{ static void Main() { for (int i = 0; i < 1000000; i++)
{ var obj = new object(); // Создаем объекты, заполняя
Gen 0 } // GC может запуститься автоматически из-за нехватки
памяти
GC.Collect(); // Явный вызов GC (не рекомендуется) }}
```

2 (4.4)

Платформенный вызов (Platform Invoke, P/Invoke) — это механизм в .NET (включая C#), который позволяет управляемому коду вызывать неуправляемые функции, реализованные в нативных библиотеках (например, DLL в Windows), написанных на языках вроде C или C++. Or.

Платформенный вызов (Platform Invoke, P/Invoke) — это механизм в .NET, который позволяет управляемому коду (C#, VB.NET и др.) вызывать неуправляемые функции из DLL-библиотек, написанных на языках вроде C или C++. ➤ Основные понятия

Управляемый код — код, выполняемый под управлением CLR (.NET). Неуправляемый код — нативный код (например, WinAPI, системные библиотеки). DLL (Dynamic Link Library) — библиотека, содержащая функции, которые можно вызывать извне

Variant 8

1.3 (The same as in 13 with the callback)

8 (2.4)

The other function is the same only in the main function it changes:

```
static void Main() {
var oddNumbers = MakeGenerator<int>(1, x =>
{
int next = x + 2; return next > 25 ? 1: next; });
// Получаем перечислитель
var enumerator = oddNumbers.GetEnumerator();
Console.WriteLine("Таблица произведений нечетных
чисел от 1 до 25:");
// Выводим 10 строк таблицы (каждая строка - произ-
ведение числа на 1-10)
for (int i = 0; i < 10; i++) {
enumerator.MoveNext();
int number = enumerator.Current;
Console.WriteLine($"{number,2} | ");
for (int j = 1; j <= 10; j++){
Console.WriteLine($"{number * j,3} ");
Console.WriteLine();}}}
```

8 (3.4)

В управляемой куче (managed heap) .NET **адрес объекта может меняться** во время работы сборщика мусора (Garbage Collector, GC). Это происходит из-за процесса **сжатия памяти** (compaction), который GC выполняет для оптимизации использования памяти.

Когда и почему меняется адрес? Во время сборки мусора:

При сборке мусора (особенно для поколений 0 и 1) CLR может перемещать живые объекты в памяти, чтобы устранить фрагментацию.

Старые адреса объектов становятся невалидными, а новые назначаются после перемещения.

При работе с Large Object Heap (LOH)

Объекты размером ≥ 85 КБ попадают в LOH, который не сжимается по умолчанию (адреса таких объектов не меняются).

Начиная с .NET 4.5.1, можно включить сжатие LOH через **GCSettings.LargeObjectHeapCompactionMode**.

8 (4.1)

```
static void Main() { int[] array = { 10, 20, 5, 2, 54, 9 };
unsafe {
fixed (int* arr_ptr = &array[0]) // Закрепляем массив и
получаем указатель {
// Пример использования указателя
for (int i = 0; i < array.Length; i++) {
Console.WriteLine(*(arr_ptr + i)); // Доступ к элементам
через указатель } } }
```

Variant 6

Variant 6 (1.1)

```
using System; using System.Threading; using System.Linq;
public class Program{ public static void Main() {
ClsA processor = new ClsA();
processor.StartProcessing(); }
public class ClsA{
private readonly Random _random = new Random();
private char[] _sharedArray; private int _iterations;
private int _currentIteration = 0;
private readonly Semaphore _semB = new Semaphore(0, 1);
private readonly Semaphore _semC = new Semaphore(0, 1);
private readonly Semaphore _semA = new Semaphore(0, 1);
private ClsB _clsB; private ClsC _clsC;
public ClsA() {
_clsB = new ClsB(this); _clsC = new ClsC(this);
_iterations = _random.Next(3, 7); // 3-6 итераций }
public void StartProcessing() {
Console.WriteLine($"Starting processing with {_iterations}
iterations");
new Thread(_clsB.Run).Start(); new Thread(_clsC.Run).Start();
for (_currentIteration = 0; _currentIteration < _iterations;
_currentIteration++) {
GenerateArray();Console.WriteLine($"\\nIteration {_currentIteration
+ 1}: Initial array: {new string(_sharedArray)}");
_semB.Release(); _semA.WaitOne(); }
_clsB.Stop(); _clsC.Stop(); _semB.Release(); _semC.Release(); }
private void GenerateArray() {
int size = _random.Next(10, 20); _sharedArray = new char[size];
for (int i = 0; i < size; i++) { if (_random.Next(2) == 0)
_sharedArray[i] = (char)_random.Next('0', '9' + 1);
else _sharedArray[i] = (char)_random.Next('A', 'Z' + 1);
} }
public char[] GetSharedArray() => _sharedArray;
public Semaphore GetSemaphoreB() => _semB;
```

```

public Semaphore GetSemaphoreC() => _semC;
public void CallbackFromC() { Console.WriteLine($"Result after
sorting: {new string(_sharedArray)}");
_semA.Release(); }
public class ClsB{
private readonly ClsA _clsA; private bool _isRunning = true;
public ClsB(ClsA clsA) { _clsA = clsA; }
public void Run() { while (_isRunning) {
_clsA.GetSemaphoreB().WaitOne(); if (!_isRunning) break;
var array = _clsA.GetSharedArray();
var digits = array.Where(c => char.IsDigit(c)).OrderBy(c =>
c).ToArray(); int digitIndex = 0;
for (int i = 0; i < array.Length; i++) {
if (char.IsDigit(array[i])) {
array[i] = digits[digitIndex++]; } }
Console.WriteLine($"B: Sorted digits: {new
string(array.Where(c => char.IsDigit(c)).ToArray())}");
_clsA.GetSemaphoreC().Release(); } }
public void Stop() => _isRunning = false;
public class ClsC{
private readonly ClsA _clsA; private bool _isRunning = true;
public ClsC(ClsA clsA) { _clsA = clsA; }
public void Run() { while (_isRunning) {
_clsA.GetSemaphoreC().WaitOne(); if (!_isRunning) break;
var array = _clsA.GetSharedArray();
var letters = array.Where(c => char.IsLetter(c)).OrderBy(c =>
c).ToArray();
int letterIndex = 0;
for (int i = 0; i < array.Length; i++){
if (char.IsLetter(array[i])) {
array[i] = letters[letterIndex++]; } }
Console.WriteLine($"C: Sorted letters: {new
string(array.Where(c => char.IsLetter(c)).ToArray())}");
_clsA.CallbackFromC(); } }
public void Stop() => _isRunning = false; }

```

6 (2.3)

```

class LambdaTest{
static void Main(string[] args) {
var teamMembers = new List<string> { "Lou Loomis", "Smoke
Porterhouse", "Danny Noonan", "Ty Webb" };
Find(teamMembers, "Dan", (member, substr) => {
// Разделяем строку на слова
var words = member.Split(' ');
// Проверяем, начинается ли
foreach (var word in words) {
if(word.StartsWith(substr,StringComparison.OrdinalIgnoreCase))
{ return true; } }
return false; }); }
static void Find(List<string> members, string substr, Func<string,
string, bool> predicate) {
foreach (var member in members) {
if (predicate(member, substr)) {
Console.WriteLine(member); } } }

```

6. (3.3)

In second variant (3.3) already answered.

6 (4.1)

1. **Отсутствие контекста unsafe:** В C# работа с указателями требует явного указания unsafe. Без ключевого слова unsafe компилятор выдаст ошибку, так как указатели не поддерживаются в управляемом коде.

2.Перемещение массива сборщиком мусора: Массивы в .NET — управляемые объекты, и сборщик мусора (GC) может перемещать их в памяти. Если указатель arr_ptr ссылается на &array[0], а GC переместит массив, указатель станет недействительным, что может привести к **неопределённому поведению** или сбою программы (например, AccessViolationException).

3. Некорректный доступ к памяти: Без закрепления массива с помощью fixed, указатель arr_ptr может указывать на некорректный адрес после перемещения массива GC. Это может вызвать: а) Чтение/запись в неверную область памяти. В)Сегментацию памяти или крах приложения.

4. Отсутствие проверки границ: При работе с указателями C# не выполняет автоматическую проверку границ массива. Если через arr_ptr обратиться к индексу за пределами массива (например, *(arr_ptr + 10)), это вызовет неопределённое поведение или сбой, так как указатель выйдет за выделенную память.

Variant 11

1.1 (the answer is already in the 6 variant number.)

2.1 (the answer is in the 2 second variant 2.1)

Что такое поколения объектов?

В .NET (и других платформах с управляемой памятью, таких как Java) **поколения объектов** — это механизм, используемый сборщиком мусора (Garbage Collector, GC) для оптимизации управления памятью. Объекты в управляемой куче (heap) делятся на категории, называемые поколениями, на основе их "возраста" (сколько сборок мусора они пережили). Это позволяет GC эффективнее освобождать память, сосредотачиваясь на "молодых" объектах, которые чаще становятся ненужными.

Какие бывают поколения в .NET?

В .NET существуют три поколения объектов:

1. **Generation 0 (Gen 0): Описание:** Сюда попадают **новые объекты**, только что созданные.
2. **Generation 1 (Gen 1): Описание:** Объекты, пережившие **одну сборку мусора** из Gen 0.
- 2.**Generation 2 (Gen 2): Описание:** Объекты, пережившие **несколько сборок мусора** (из Gen 1).
- 4.4 (the answer is in the 2 variant 4.4)

Variant 4

4 (1.4)

```

using System;using System.Threading;using System.Linq;
public class Program{ public static void Main() {
var processor = new ClsA(); processor.StartProcessing(); } }
public class ClsA{
private readonly Random _random = new Random();
private int[] _sharedArray; private int _iterations;
private int _currentIteration = 0;
private readonly Semaphore _semB = new Semaphore(0, 1);
private readonly Semaphore _semC = new Semaphore(0, 1);
private readonly Semaphore _semA = new Semaphore(0, 2);
private ClsB _clsB; private ClsC _clsC;

```

```

public ClsA() {
    _clsB = new ClsB(this);    _clsC = new ClsC(this);
    _iterations = _random.Next(3, 7); // 3-6 итераций }
    public void StartProcessing() {
        Console.WriteLine($"Starting processing with { _iterations}
iterations");
        new Thread(_clsB.Run).Start();    new Thread(_clsC.Run).Start();
        for (_currentIteration = 0; _currentIteration < _iterations;
_currentIteration++) {
            GenerateArray();
            Console.WriteLine($"\\nIteration { _currentIteration + 1}: Generated
array: {string.Join(", ", _sharedArray)}");
            _semB.Release(); _semA.WaitOne(); _semC.Release();
            _semA.WaitOne();    }
        _clsB.Stop(); _clsC.Stop();    _semB.Release(); _semC.Release();    }
        private void GenerateArray() {    int size = _random.Next(5, 15);
        _sharedArray = new int[size];    for (int i = 0; i < size; i++)    {
            _sharedArray[i] = _random.Next(-50, 50);    }    }
        public int[] GetSharedArray() => _sharedArray;
        public Semaphore GetSemaphoreB() => _semB;
        public Semaphore GetSemaphoreC() => _semC;
        public void SignalFromB(int maxNegative) {
            Console.WriteLine($"Max negative from B: {maxNegative}");
            _semA.Release();    }
        public void SignalFromC(int maxCount) {
            Console.WriteLine($"Max count from C: {maxCount}");
            _semA.Release();    }
    }
    public class ClsB{
        private readonly ClsA _clsA;    private bool _isRunning = true;
        public ClsB(ClsA clsA) {    _clsA = clsA;    }
        public void Run() {    while (_isRunning)    {
            _clsA.GetSemaphoreB().WaitOne();    if (!_isRunning) break;
            var array = _clsA.GetSharedArray();
            int maxNegative = array.Where(x => x < 0).DefaultIfEmpty(0).Max();
            _clsA.SignalFromB(maxNegative);    }    }
        public void Stop() => _isRunning = false;
    }
    public class ClsC{
        private readonly ClsA _clsA;    private bool _isRunning = true;
        public ClsC(ClsA clsA) {    _clsA = clsA;    }
        public void Run() {    while (_isRunning)    {
            _clsA.GetSemaphoreC().WaitOne(); if (!_isRunning) break;
            var array = _clsA.GetSharedArray();
            int max = array.Max(); int maxCount = array.Count(x => x == max);
            _clsA.SignalFromC(maxCount);    }    }
        public void Stop() => _isRunning = false;    }
}

```

4 (2.3)

```

interface Animal {
    legCount(): number; }
class Lion implements Animal {
    legCount() { return 4; } }
class Mouse implements Animal {
    legCount() { return 4; } }
class Snake implements Animal {
    legCount() { return 0; } }
function animalLegCount(animals: Animal[]) {
    return animals.map(a => a.legCount());
}

```

4 (3.1) (The answer is in the 13 variant with call back)

4(4.2) (The answer is also there in 13 with call back)

12 variant

12 (1.2)

```

using System; using System.Threading; using System.Linq;
public class Program {    public static void Main()    {
    var processor = new ClsA();    processor.StartProcessing();    } }
public class ClsA {private readonly Random _random = new Random();
    private int[] _sharedArray;    private int _iterations;
    private int _currentIteration = 0;
    private readonly Semaphore _semB = new Semaphore(0, 1);
    private readonly Semaphore _semC = new Semaphore(0, 1);
    private readonly Semaphore _semA = new Semaphore(0, 2);
    private ClsB _clsB;    private ClsC _clsC;
    public ClsA() {    _clsB = new ClsB(this);    _clsC = new ClsC(this);
    _iterations = _random.Next(3, 7); // 3-6 итераций    }
    public void StartProcessing() {    Console.WriteLine($"Starting
processing with { _iterations} iterations");
        new Thread(_clsB.Run).Start();    new Thread(_clsC.Run).Start();
        for (_currentIteration = 0; _currentIteration < _iterations;
_currentIteration++)    {
            GenerateArray();
            Console.WriteLine($"\\nIteration { _currentIteration + 1}:
Generated array: {string.Join(", ", _sharedArray)}");
            _semB.Release();    _semC.Release();
            _semA.WaitOne(); // От ClsB _semA.WaitOne(); // От ClsC    }
        _clsB.Stop(); _clsC.Stop(); _semB.Release(); _semC.Release();    }
        private void GenerateArray()    {
            int size = _random.Next(5, 11);    _sharedArray = new int[size];
            for (int i = 0; i < size; i++)    {
                _sharedArray[i] = _random.Next(1, 10);    }    }
        public int[] GetSharedArray() => _sharedArray;
        public Semaphore GetSemaphoreB() => _semB;
        public Semaphore GetSemaphoreC() => _semC;
        public void CallbackFromB(double arithmeticMean)    {
            Console.WriteLine($"Arithmetic mean from B:
{arithmeticMean:F2}");    _semA.Release();    }
        public void CallbackFromC(double geometricMean)    {
            Console.WriteLine($"Geometric mean from C:
{geometricMean:F2}");    _semA.Release();    }    }
    public class ClsB {
        private readonly ClsA _clsA;    private bool _isRunning = true;
        public ClsB(ClsA clsA) {    _clsA = clsA;    }
        public void Run() {    while (_isRunning)    {
            _clsA.GetSemaphoreB().WaitOne();    if (!_isRunning) break;
            var array = _clsA.GetSharedArray();
            double arithmeticMean = array.Average();
            _clsA.CallbackFromB(arithmeticMean);    }    }
        public void Stop() => _isRunning = false;    }
    public class ClsC {
        private readonly ClsA _clsA;    private bool _isRunning = true;
        public ClsC(ClsA clsA) {    _clsA = clsA;    }
        public void Run() {    while (_isRunning)    {
            _clsA.GetSemaphoreC().WaitOne() if (!_isRunning) break;
            var array = _clsA.GetSharedArray();
            double product = 1;    foreach (var num in array)    {
                product *= num;    }
        }
    }
}

```

```

        double geometricMean = Math.Pow(product, 1.0 /
array.Length);
        _clsA.CallbackFromC(geometricMean);    } }
    public void Stop() => _isRunning = false; }

```

12 (2.5 2.3)

```

class LambdaTest {
    static void Main(string[] args) {
        var teamMembers = new List<string> { "Lou Loomis", "Smoke
Porterhouse", "Danny Noonan", "Ty Webb" };
        Find(teamMembers, "Dan", (member, substr) => {
            var words = member.Split(' ');
            // Проверяем, что ни одно слово не начинается с заданной
            foreach (var word in words) {
                if(word.StartsWith(substr,StringComparison.OrdinalIgnoreCase))
            { return false; } }
            return true; }); }
        static void Find(List<string> members, string substr, Func<string,
string, bool> predicate) { foreach (var member in members) {
            if (predicate(member, substr)) { Console.WriteLine(member); }}} }

```

12 (3.5) (The answer is in the 8 variant 3.4)

12 (4.1) (The answer is in the 8 variant 4.1)

Variant 10

10 (1.5) Did not answer, according to luck

10 (2.5)

```

public class Employee
{    public int ID { get; set; }    public string FullName { get; set; } }

```

// Отдельный класс для работы с БД

```

public class EmployeeRepository {
    public bool Add(Employee emp) {
        // Вставка данных сотрудника в таблицу БД
        return true; } }

```

// Отдельный класс для генерации отчетов

```

public class EmployeeReportGenerator{
    public void GenerateReport(Employee emp) {
        // Генерация отчета по деятельности сотрудника }
    }

```

10 (3.3) (answer in the 2 variant 2 (3.3))

10 (4.3)

В C# при работе с указателями в небезопасном (unsafe) контексте определение длины строки при индексировании указателя зависит от типа строки (например, string в управляемом коде или С-строка в неуправляемом коде) и способа её представления в памяти.

Определение длины строки при индексировании указателя

1. Управляемая строка через массив символов:

a)Строку можно преобразовать в массив char[], закрепить его в памяти с помощью fixed, и получить указатель.

b)Длина строки доступна через свойство string.Length или через длину массива символов (char[].Length).

c)При индексировании указателя важно вручную проверять границы, чтобы не выйти за пределы массива.

2. С-строка (нуль-терминированная):

a)Если строка представлена как массив байт или символов в неуправляемой памяти (например, через IntPtr или char*), длина определяется подсчетом символов до \0.

b) Это типично при взаимодействии с нативными библиотеками через P/Invoke.